

Emulation of Complex Matrix Multiplication based on the Chinese Remainder Theorem

Yuki Uchino*, Qianxiang Ma[†], Toshiyuki Imamura[‡], Katsuhisa Ozaki[§], and Patrick Lars Gutsche[¶]

*RIKEN Center for Computational Science, Hyogo, Japan
yuki.uchino.fe@riken.jp

[†]RIKEN Center for Computational Science, Hyogo, Japan
qianxiang.ma@riken.jp

[‡]RIKEN Center for Computational Science, Hyogo, Japan
imamura.toshiyuki@riken.jp

[§]Department of Mathematical Science
Shibaura Institute of Technology, Saitama, Japan
ozaki@sic.shibaura-it.ac.jp

[¶]Computer Science Department
Ecole Normale Supérieure de Lyon, Lyon, France
patrick.gutsche@ens-lyon.fr

Abstract—Modern computing architectures feature low-precision matrix multiplication units that achieve substantially higher throughput than their high-precision counterparts. Motivated by this architectural trend, the emulation of high-precision matrix multiplication using low-precision hardware has attracted significant interest in the high-performance computing community. Ozaki, Uchino, and Imamura introduced the Ozaki-II scheme as a general framework for emulating matrix multiplication. Building on this framework, Uchino, Ozaki, and Imamura developed high-performance and power-efficient techniques for emulating single- and double-precision real matrix multiplication on INT8 matrix engines. Extending this line of research, the present study proposes high-performance emulation methods for single- and double-precision complex matrix multiplication on INT8 matrix engines, based on the Ozaki-II scheme. On an NVIDIA B200 GPU, the proposed methods achieve $4.0\times$ – $5.6\times$ and $4.4\times$ – $6.5\times$ speedups over the native single- and double-precision complex matrix multiplication routines from cuBLAS, respectively, for sufficiently large problem sizes. When lower accuracy than that of the standard routine is acceptable, the proposed methods can operate at even higher speed. Conversely, with only a modest increase in computation time, they can also deliver higher accuracy than the standard routines. These properties suggest that the proposed approach has the potential to serve as a default algorithm across a wide range of applications.

Index Terms—emulation, mixed-precision computing, complex matrix multiplication, high-performance computing

I. INTRODUCTION

A. Background

General-purpose computing architectures have increasingly adopted low-precision matrix multiplication units in Graphics Processing Units (GPUs), Tensor Processing Units (TPUs), and Neural Processing Units (NPU). Representative examples include Tensor Cores in NVIDIA GPUs [1], Matrix Cores in AMD GPUs [2], the MXU in Google TPUs [3], and the MAC array in Intel NPUs [4]. These units offer substantially higher

arithmetic throughput than high-precision units and have become central components of modern accelerator design. Table I shows the theoretical throughput performance in TFLOPS or TOPS of AMD [5]–[9] and NVIDIA GPUs [10]–[13]. In recent years, such units have also begun to appear as standard features even in laptop-class processors [14], [15], further broadening their availability. While these units were originally motivated by machine-learning workloads, their architectural characteristics also present an opportunity to accelerate high-precision computations by emulating them on low-precision hardware. Consequently, emulation-based high-precision matrix multiplication has emerged as an active research topic in the high-performance computing community [16]–[22].

TABLE I
TFLOPS AND TOPS OF AMD AND NVIDIA GPUS [5]–[13]

GPU	FP64	FP32	FP16	FP8	INT8
Instinct MI300A CDNA3	122.6	122.6	980.6	1961.2	1961.2
Instinct MI300X CDNA3	163.4	163.4	1307.4	2614.9	2614.9
Instinct MI325X CDNA3	163.4	163.4	1307.4	2614.9	2614.9
Instinct MI350X CDNA4	72.1	144.2	2306.9	4613.7	4614
Instinct MI355X CDNA4	78.6	157.3	2516.6	5033.2	5033.2
Ampere A100 SXM	19.5	19.5	312	624	624
Hopper H100 SXM	66.9	66.9	989.4	1978.9	1978.9
Blackwell B200 SXM	37	75	2250	4500	4500
Blackwell Ultra B300 SXM	1.2	75	2250	4500	150
GeForce RTX 5080 Blackwell	0.88	56.3	112.6	450.2	450.2

B. Contributions

A prior line of work introduced the Ozaki-I [23]–[28] and Ozaki-II [29], [30] schemes as frameworks for emulating high-precision matrix multiplication using low-precision matrix engines, demonstrating high performance and power efficiency on contemporary GPUs. In parallel, the Ozaki-I scheme has recently been incorporated into cuBLAS (from

version 13.0 Update 2), and this implementation supports complex matrix multiplication. However, for real-valued matrix multiplication, previous studies have reported that the Ozaki-II scheme achieves higher performance than Ozaki-I. This suggests that a complex extension of the Ozaki-II scheme would likewise outperform the complex Ozaki-I approach. Despite this expectation, no complex-valued formulation of Ozaki-II has been developed to date. This absence is notable because complex matrix multiplication is a key computational kernel in many HPC applications, and its performance can dominate the overall runtime of large-scale scientific simulations.

To address this gap, the present work extends the Ozaki-II scheme to single- and double-precision complex matrix multiplication (CGEMM and ZGEMM, respectively). We develop emulation algorithms for CGEMM and ZGEMM by combining the Ozaki-II scheme with a Karatsuba-based formulation of complex arithmetic. The resulting methods maintain numerical accuracy comparable to standard CGEMM and ZGEMM while effectively exploiting the high throughput of low-precision matrix engines. In addition, several improvements applicable to existing SGEMM and DGEMM emulation techniques are introduced, further enhancing the efficiency of the Ozaki schemes in the real domain. The proposed algorithms are released as a portable library supporting both CUDA and HIP backends, enabling execution on NVIDIA GPUs and AMD GPUs. Performance is evaluated on an NVIDIA GH200 Grace Hopper Superchip, an NVIDIA B200 Blackwell GPU, an NVIDIA GeForce RTX 5080 Blackwell GPU, and an AMD Instinct MI300X GPU, and compared against cuBLAS and hipBLAS implementations. For sufficiently large matrix sizes, the proposed methods surpass both vendor-optimized libraries and, in particular, achieve higher throughput than the cuBLAS Ozaki-I-based ZGEMM emulation, demonstrating the performance potential of Ozaki-II-based emulation for CGEMM and ZGEMM. We also provide performance models for the proposed methods, and the predicted throughput closely aligns with the measured results across all tested configurations.

The remainder of this paper is organized as follows. Section II introduces the Ozaki-II scheme for real-valued matrix multiplication. Section III presents the proposed efficient methods for emulating CGEMM and ZGEMM. Section IV shows the numerical results comparing the proposed emulation with the vendor-optimized GEMM routines. Section V presents the concluding remarks. Note that in this paper, we do not consider matrix shapes for which performance becomes memory-bound, such as small matrices or highly skinny matrices, because low-precision matrix engines do not achieve high throughput on such workloads.

II. OZAKI-II SCHEME FOR REAL MATRIX MULTIPLICATION

The Ozaki-II scheme is an emulation method of matrix multiplication based on a large-integer multiplication technique using the Chinese Remainder Theorem (CRT). The scheme for matrix multiplication of $A \in \mathbb{R}^{m \times k}$ and $B \in \mathbb{R}^{k \times n}$ consists of the following three steps:

- 1) Convert A and B to $A' \in \mathbb{Z}^{m \times k}$ and $B' \in \mathbb{Z}^{k \times n}$, respectively, by applying diagonal scaling and truncation.
- 2) Compute the integer matrix product $C' \approx A'B'$ based on the CRT.
- 3) Convert C' back to floating-point format $C \in \mathbb{R}^{m \times n}$ by inversely scaling.

The integer matrix multiplication in step 2 is performed by decomposing the elements of A' and B' into residues with respect to a predefined set of pairwise coprime moduli. Let $p_1, p_2, \dots, p_N \in \mathbb{N}$ denote these moduli and define $P := \prod_{\ell=1}^N p_\ell$. Let $\text{mod}(x, p_\ell)$ for $x \in \mathbb{Z}$ denote the symmetric remainder of x/p_ℓ , i.e., $\text{mod}(x, p_\ell) = x - p_\ell \cdot \text{round}(x/p_\ell)$, where $\text{round}(\cdot)$ rounds its input to the nearest integer. Each element of A' and B' is mapped to its residue representation modulo each p_ℓ as

$$(A'_\ell)_{ij} := \text{mod}(a'_{ij}, p_\ell), \quad (B'_\ell)_{ij} := \text{mod}(b'_{ij}, p_\ell). \quad (1)$$

Then, the corresponding matrix products are computed independently for each p_ℓ as

$$C'_\ell := A'_\ell B'_\ell. \quad (2)$$

This procedure yields a set of partial results

$$C'_\ell \equiv A'B' \pmod{p_\ell}, \quad \ell = 1, \dots, N. \quad (3)$$

The final integer matrix C' is then reconstructed from C'_ℓ by applying the CRT as

$$C' := \sum_{\ell=1}^N \frac{P}{p_\ell} q_\ell \cdot C'_\ell.$$

Then, we obtain

$$C' \equiv A'B' \pmod{P},$$

namely, $c'_{ij} = \text{mod}((A'B')_{ij}, P) + zP$ for $z \in \mathbb{Z}$. If the condition

$$2 \sum_{h=1}^k |a'_{ih}| |b'_{hj}| < P \quad (4)$$

is satisfied for all (i, j) pairs, the possible value of c'_{ij} is only $\text{mod}((A'B')_{ij}, P)$ and the CRT guarantees that the reconstructed C' is unique. Therefore, the scaling vector used in step 1 must be chosen such that the bound (4) is ensured for A' and B' . Increasing the number of moduli p_ℓ enlarges P , thereby relaxing the uniqueness condition. However, this also increases the number of matrix multiplications required, leading to higher computational cost.

In previous work [30], an algorithm was proposed in which each modular matrix multiplication (3) is executed using INT8 matrix engines. Algorithm 1 shows the outline. Hereafter, we assume that $k \leq 2^{17}$ to perform INT8 matrix multiplication without error. In [30], step iv at the line 5 of Algorithm 1 uses the standard (non-symmetric) modulo operation that returns values in $[0, p_\ell)$ and E'_ℓ are UINT8 matrices. In DGEMM

emulation, step v at the line 5 evaluates $S = \sum_{\ell=1}^N P/p_\ell \cdot q_\ell \cdot E'_\ell$ by the unevaluated sum $S_1 + S_2$ as

$$S \approx S_1 + S_2, \quad S_1 := \sum_{\ell=1}^N s_{\ell 1} E'_\ell, \quad S_2 := \sum_{\ell=1}^N s_{\ell 2} E'_\ell, \quad (5)$$

where $s_{\ell 1}$ contains the upper $(53 - 8 - \lceil \log_2 N \rceil)$ bits of $P/p_\ell \cdot q_\ell$, $s_{\ell 2}$ stores the remaining lower bits rounded to double precision, and $s_{\ell 1} + s_{\ell 2} \approx P/p_\ell \cdot q_\ell$. In (5), the value of $s_{\ell 1}$ is limited by the precision of the double-precision significand. Since each element of E'_ℓ is an 8-bit unsigned integer, the product $s_{\ell 1} E'_\ell$ requires 8 bits of significand to represent the contribution of E'_ℓ . Moreover, accumulating N such terms requires an additional $\lceil \log_2 N \rceil$ bits to avoid overlap between partial sums. Thus, the high-order part $s_{\ell 1}$ may occupy at most $(53 - 8 - \lceil \log_2 N \rceil)$ bits of precision. This bit allocation ensures that all products $s_{\ell 1} E'_\ell$ can be accumulated in S_1 without rounding error. In contrast, the present work adopts a symmetric modulo operation in step iv at the line 5, ensuring that each E'_ℓ becomes INT8 from UINT8. Consequently, the upper part $s_{\ell 1}$ of $P/p_\ell \cdot q_\ell$ can use $(53 - 7 - \lceil \log_2 N \rceil)$ bits instead of $(53 - 8 - \lceil \log_2 N \rceil)$ bits, gaining one additional effective bit compared with the previous method. This slight but meaningful increase in representable precision improves the numerical robustness of the reconstruction step without changing the overall algorithmic structure.

Algorithm 1 Outline of Ozaki-II scheme using INT8 matrix engines for real matrix multiplication [30]. The function $\text{trunc}(\cdot)$ returns the integer part of its input.

Input: $A \in \mathbb{R}^{m \times k}$, $B \in \mathbb{R}^{k \times n}$, $N \in \mathbb{N}$

Output: $C \approx AB \in \mathbb{R}^{m \times n}$

- 1: Determine the moduli $p_\ell \leq 256$ for $\ell = 1, \dots, N$.
- 2: Determine modular multiplicative inverses q_ℓ of P/p_ℓ modulo p_ℓ for $\ell = 1, \dots, N$.
- 3: Determine the scaling vectors $\mu \in \mathbb{R}^m$ and $\nu \in \mathbb{R}^n$.
- 4: Convert A and B to $A' \in \mathbb{Z}^{m \times k}$ and $B' \in \mathbb{Z}^{k \times n}$:
 - i) $a'_{ij} := \text{trunc}(a_{ij} \cdot \mu_i)$
 - ii) $b'_{ij} := \text{trunc}(b_{ij} \cdot \nu_j)$
- 5: Compute $C' \approx A'B'$ based on the CRT:
 - i) $(A'_\ell)_{ij} := \text{mod}(a'_{ij}, p_\ell)$ for $\ell = 1, \dots, N$
 - ii) $(B'_\ell)_{ij} := \text{mod}(b'_{ij}, p_\ell)$ for $\ell = 1, \dots, N$
 - iii) $D'_\ell := A'_\ell B'_\ell$ using INT8-GEMM for $\ell = 1, \dots, N$
 - iv) $(E'_\ell)_{ij} := \text{mod}(d'_{ij}, p_\ell)$ for $\ell = 1, \dots, N$
 - v) $S := \sum_{\ell=1}^N P/p_\ell \cdot q_\ell \cdot E'_\ell$
 - vi) $c'_{ij} := \text{mod}(s_{ij}, P)$
- 6: Convert C' back to floating-point format $C \in \mathbb{R}^{m \times n}$:
 - i) $C := \text{diag}(\mu)^{-1} C' \text{diag}(\nu)^{-1}$

To satisfy the condition (4) efficiently, the scaling vectors used in step 1 must be determined appropriately. Previous work introduced two strategies for computing this scaling: a fast mode and an accurate mode. The fast mode estimates an upper bound of $\sum_{h=1}^k |a'_{ih}| |b'_{hj}|$ via the Cauchy–Schwarz inequality and determines the scaling vector analytically based

on this estimate. This approach incurs minimal preprocessing cost, but the bound obtained from the Cauchy–Schwarz inequality tends to be overly conservative. As a consequence, a larger P (i.e., more moduli) is often required to satisfy the uniqueness condition, increasing the computational workload in step 2. In contrast, the accurate mode estimates the upper bound by performing an auxiliary matrix multiplication using INT8 matrix engines. Although this mode requires additional computation during step 1, the direct estimation yields a much tighter bound than that obtained via the Cauchy–Schwarz inequality. This typically enables the use of fewer moduli, reducing the number of modular matrix multiplications and potentially improving overall performance.

III. PROPOSED METHODS

This section presents the extension of the Ozaki-II scheme to complex matrix multiplication, targeting high-performance execution on INT8 matrix engines. Let $A = A_R + iA_I \in \mathbb{C}^{m \times k}$ for $A_R, A_I \in \mathbb{R}^{m \times k}$ and $B = B_R + iB_I \in \mathbb{C}^{k \times n}$ for $B_R, B_I \in \mathbb{R}^{k \times n}$, and we compute $C = C_R + iC_I := AB$, where $C_R, C_I \in \mathbb{R}^{m \times n}$. A standard approach to handle complex matrix multiplication is to employ the following representation:

$$\hat{M} := \begin{pmatrix} M_R & -M_I \\ M_I & M_R \end{pmatrix} \quad (6)$$

for $M \in \{A, B, C\}$. This representation forms the basis for both steps 1 and 2 in the complex Ozaki-II scheme.

A. INT8 Complex Matrix Multiplication

Using (6), C_R and C_I are obtained by

$$\begin{pmatrix} C_R \\ C_I \end{pmatrix} = \begin{pmatrix} A_R & -A_I \\ A_I & A_R \end{pmatrix} \begin{pmatrix} B_R \\ B_I \end{pmatrix} \quad (7)$$

or

$$\begin{pmatrix} C_I & C_R \end{pmatrix} = \begin{pmatrix} A_I & A_R \end{pmatrix} \begin{pmatrix} B_R & -B_I \\ B_I & B_R \end{pmatrix}. \quad (8)$$

The formulations in (7) and (8) allow complex matrix multiplication to be computed using a single real GEMM applied to an expanded matrix. For the formulations, the applicability condition on the inner dimension changes from $k \leq 2^{17}$ to $k \leq 2^{16}$. Another standard approach to handle complex matrix multiplication is the Karatsuba algorithm. The Karatsuba algorithm expresses the complex product using three real matrix multiplications as

$$C_R := D - E, \quad C_I := F - D - E \quad (9)$$

for

$$\begin{cases} D := A_R B_R, \\ E := A_I B_I, \\ F := (A_R + A_I)(B_R + B_I), \end{cases} \quad (10)$$

without forming enlarged matrices. We evaluated the three approaches based on (7), (8), and (10) on INT8 matrix engines. Note that both $A_R + A_I$ and $B_R + B_I$ in (10) are reduced to the INT8 range by applying the modulo operation before forming the products. The performance results are summarized

in Fig. 1. In addition to these three methods, we also tested a blocked variant of (10). This variant applies the Karatsuba formulation but partitions the output dimension n into blocks of size 8192, thereby limiting the working set of each INT8 GEMM and preventing performance degradation for very large n . For problem sizes up to approximately 2000, the methods based on (7) and (8) are slightly faster. When the size exceeds 3000, both the original (10) and the blocking variant of (10) achieve $1.2\times$ – $1.4\times$ higher throughput. However, for sizes above 15000, the performance of the unblocked (10) deteriorates, while the blocked version maintains stable throughput. Similar results were observed on other GPUs, such as a GH200 and a B200. Motivated by these observations, the proposed complex Ozaki-II scheme adopts the Karatsuba method with n -blocking for all modular INT8 complex matrix multiplications in step 2.

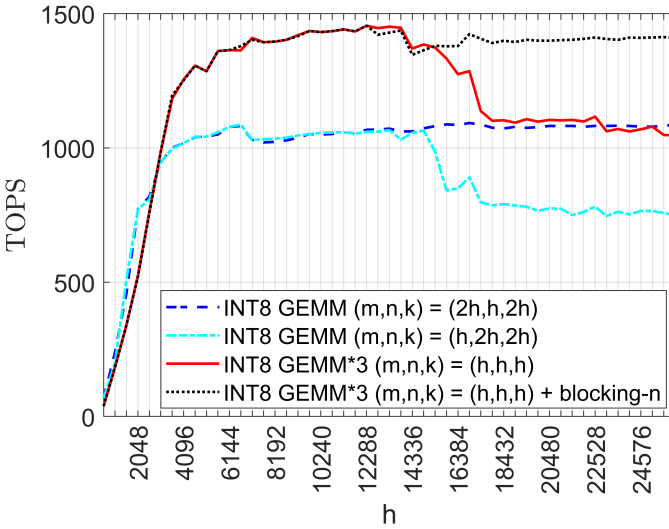


Fig. 1. Performance comparison of four INT8-based GEMM strategies on NVIDIA H100 NVL with CUDA Toolkit 12.8.61. The first two methods use a single INT8 matrix multiplication with dimensions $(2h, h, 2h)$ and $(h, 2h, 2h)$, respectively. The third method performs three INT8 multiplications of size (h, h, h) , while the fourth applies the same three-multiplication scheme but introduces blocking along the n dimension.

B. Scaling Vector Determination

First, we convert $\hat{A}$ and $\hat{B}$ to integer matrices $\hat{A}'$ and $\hat{B}'$ as

$$\hat{a}'_{ij} := \text{trunc}(\hat{a}_{ij} \cdot \mu_i), \quad \hat{b}'_{ij} := \text{trunc}(\hat{b}_{ij} \cdot \nu_j).$$

We also consider two strategies: a fast mode and an accurate mode. From $|\hat{a}'_{ij}| \leq |\hat{a}_{ij}| \cdot \mu_i$ and $|\hat{b}'_{ij}| \leq |\hat{b}_{ij}| \cdot \nu_j$,

$$\sum_{h=1}^k |\hat{a}'_{ih}| |\hat{b}'_{hj}| \leq \mu_i \nu_j \cdot \sum_{h=1}^k |\hat{a}_{ih}| |\hat{b}_{hj}|.$$

In the fast mode, using the Cauchy–Schwarz inequality yields

$$\sum_{h=1}^k |\hat{a}_{ih}| |\hat{b}_{hj}| \leq \|\hat{a}_{i,:}\|_2 \|\hat{b}_{:,j}\|_2.$$

From (6), the row norms of $\hat{A}$ satisfy $\|\hat{a}_{i,:}\|_2 = \|\hat{a}_{i+m,:}\|_2$ for $1 \leq i \leq m$, and the column norms of $\hat{B}$ satisfy $\|\hat{b}_{:,j}\|_2 =$

$\|\hat{b}_{:,j+n}\|_2$ for $1 \leq j \leq n$. Thus, the scaling values of rows i and $i+m$ of $\hat{A}$ are identical, and the same holds for columns j and $j+n$ of $\hat{B}$. Consequently, the scaling vectors reduce to $\mu \in \mathbb{R}^m$ and $\nu \in \mathbb{R}^n$. The elements of μ and ν can then be determined in the same manner as in the real-valued Ozaki-II scheme [30], i.e.,

$$\mu_i \leq 2^{\lfloor \frac{\log_2(P-1)-1}{2} - 1 - \max(1, 0.5 \cdot \log_2 \sum_h \hat{a}_{ih}^2) \rfloor - \lfloor \log_2 \max_h |\hat{a}_{ih}| \rfloor},$$

$$\nu_j \leq 2^{\lfloor \frac{\log_2(P-1)-1}{2} - 1 - \max(1, 0.5 \cdot \log_2 \sum_h \hat{a}_{jh}^2) \rfloor - \lfloor \log_2 \max_h |\hat{b}_{hj}| \rfloor}.$$

For any real number $x \in \mathbb{R}$, $\lfloor \log_2 |x| \rfloor$ represents the Unit in the First Place (UFP) of x , and thus it can be obtained directly using the `ilogb(·)` function. Following [30], the quantity $(\log_2(P-1) - 1)/2 - 1$ is precomputed as $\mathcal{P}'_{fast} := \text{single}(\log_2(P-1)/2 - 1.5)$. Furthermore, $\log_2 \|\hat{a}_{i,:}\|_2$ and $\log_2 \|\hat{b}_{:,j}\|_2$ can be upper-bounded by $0.51 \cdot _log2f(\sum_{h=1}^{2k} \hat{a}_{ih}^2)$ and $0.51 \cdot _log2f(\sum_{h=1}^{2k} \hat{b}_{hj}^2)$, respectively, using floating-point arithmetic in round-up mode. Here, `\_log2f` denotes the fast built-in base-2 logarithm function provided by both the CUDA Math API and the HIP math API. According to NVIDIA's official CUDA documentation, the maximum absolute error of `\_log2f(x)` is 2^{-22} for $x \in [0.5, 2]$, and at most 2 Units in the Last Place (ULP) otherwise [31]. The definition of ULP used here follows Muller [32]. In addition, the AMD HIP documentation states that the maximum absolute error of `\_log2f(x)` is 1 ULP for $x \in [10^{-6}, 10^6]$ [33], although the precise definition of ULP is not specified. Based on these specifications, we assume the bound $_log2f(x) \leq (\log_2 x)/(1 - 4u)$, where u is the unit roundoff of FP32 arithmetic. If x is not sufficiently close to 1, it is first scaled to $x' := x/2^{\lfloor \log_2 |x| \rfloor}$, after which the logarithm is evaluated as `\_log2f(x') + \lfloor \log_2 |x| \rfloor`. Based on the above considerations, for $\delta := \text{single}(0.5/(1 - 4u))$ computed in round-down mode, the scaling vectors μ and ν are determined as follows:

$$\mu_i := 2^{\lfloor \mathcal{P}'_{fast} - \max(1, \delta \cdot _log2f(\sum_h \hat{a}_{ih}^2)) \rfloor - \lfloor \log_2 \max_h |\hat{a}_{ih}| \rfloor}, \quad (11)$$

$$\nu_j := 2^{\lfloor \mathcal{P}'_{fast} - \max(1, \delta \cdot _log2f(\sum_h \hat{b}_{hj}^2)) \rfloor - \lfloor \log_2 \max_h |\hat{b}_{hj}| \rfloor}, \quad (12)$$

where $\mathcal{P}'_{fast} - \delta \cdot x$ for $x \in \mathbb{R}$ is computed using fused multiply-add (fma) operation in round-down mode.

In accurate mode, first, we apply diagonal scaling to A_R , A_I , B_R , and B_I so that the integer parts of the maximum absolute values in each row of A_R and A_I , and in each column of B_R and B_I , fit within 6 bits. Specifically, we scale them as $\text{diag}(\bar{\mu})A_R$, $\text{diag}(\bar{\mu})A_I$, $B_R \text{diag}(\bar{\nu})$, $B_I \text{diag}(\bar{\nu})$, and then round them upward using the ceiling operation to obtain 7-bit or smaller integer matrices $\bar{A}_R$, $\bar{A}_I$, $\bar{B}_R$, $\bar{B}_I$, respectively. These matrices satisfy

$$\begin{aligned} |\bar{\mu}_i(A_R)_{ij}| &\leq (\bar{A}_R)_{ij}, & |\bar{\mu}_i(A_I)_{ij}| &\leq (\bar{A}_I)_{ij}, \\ |(B_R)_{ij}\bar{\nu}_j| &\leq (\bar{B}_R)_{ij}, & |(B_I)_{ij}\bar{\nu}_j| &\leq (\bar{B}_I)_{ij}. \end{aligned}$$

After this preprocessing, we compute the upper bound $\bar{C}_R + i\bar{C}_I$ of the INT8 matrix product $(\bar{A}_R + i\bar{A}_I)(\bar{B}_R + i\bar{B}_I)$ as

follows:

$$\begin{aligned}\bar{C}_I &:= \bar{A}_I \bar{B}_R + \bar{A}_R \bar{B}_I, \\ \bar{C}_R &:= \bar{C}_I + (\bar{A}_R - \bar{A}_I)(\bar{B}_R - \bar{B}_I) = \bar{A}_R \bar{B}_R + \bar{A}_I \bar{B}_I,\end{aligned}$$

where no computing error occurs in $\bar{A}_R - \bar{A}_I$ and $\bar{B}_R - \bar{B}_I$ in INT8. Then, the elements of μ and ν can be determined in the same manner as in the real-valued Ozaki-II scheme [30], i.e.,

$$\mu_i := \bar{\mu}_i \cdot 2^{\lfloor \mathcal{P}'_{accu} - \delta \cdot \log_2(\max_h \max((\bar{C}_R)_{ih}, (\bar{C}_I)_{ih})) \rfloor}, \quad (13)$$

$$\nu_j := \bar{\nu}_j \cdot 2^{\lfloor \mathcal{P}'_{accu} - \delta \cdot \log_2(\max_h \max((\bar{C}_R)_{hj}, (\bar{C}_I)_{hj})) \rfloor}, \quad (14)$$

where $\mathcal{P}'_{accu} := \text{single}(\log_2(P-1)/2 - 0.5)$, and $\mathcal{P}'_{accu} - \delta \cdot x$ for $x \in \mathbb{R}$ is computed using fma in round-down mode.

C. Performance Analysis

To analyze the performance characteristics of the proposed method, we employ a simple yet expressive performance model that captures both computation and memory-transfer costs. Let b denote the peak sustained memory bandwidth (in Byte/s) and p denote the peak throughput of the INT8 matrix-multiplication engine (in OPS). For a given operation, the memory-bound execution time is estimated as the required data movement divided by b , while the compute-bound time is obtained as the arithmetic workload divided by p . Because even memory-bound kernels incur a non-negligible amount of arithmetic overhead, we include a correction term c .

1) *Fast Mode*: Let the input matrices $A \in \mathbb{C}^{m \times k}$ and $B \in \mathbb{C}^{k \times n}$ be double precision. In step 1, both A and B are loaded twice: once for computing the rowwise maximum absolute values of $\hat{A}$ and the columnwise maximum absolute values of $\hat{B}$, and once for computing the residues of $\hat{A}$ and $\hat{B}$. Afterward, the scaling vectors and the INT8 matrices are stored. Each scaling factor is a power of two, so only its exponent needs to be stored as an INT16. Including the correction term c , which accounts for arithmetic overhead even in memory-bound operations, the total execution time of step 1 can be estimated as $((32 + 3N + c)k + 2)(m + n)/b$. In step 2, the INT8 GEMM kernels are executed $3N$ times. The resulting $3N$ INT32 matrices are then loaded and converted into N INT8 complex matrices. Accounting for computational overhead c , the execution time of step 2 is estimated as $6Nmnk/p + (14N + c)mn/b$. In step 3, the N INT8 complex matrices are loaded, and the CRT accumulation and final reduction are performed. The scaling vectors are then loaded, the inverse scaling is applied, and finally the resulting double-precision complex matrix is stored. With the correction term c , the execution time of step 3 is estimated as $(2(m + n) + (2N + 16 + c)mn)/b$.

Combining the costs derived for steps 1, 2, and 3, the total execution time of the proposed method can be estimated as the sum of its memory-bound and compute-bound components.

Aggregating all terms yields the following expression for the overall execution time:

$$\frac{((3N + 32 + c)k + 4)(m + n) + (16N + 16 + 2c)mn}{b} + \frac{6Nmnk}{p}.$$

Similarly, when both A and B are single-precision complex matrices, the total execution time can be estimated as

$$\frac{((3N + 16 + c)k + 4)(m + n) + (16N + 8 + 2c)mn}{b} + \frac{6Nmnk}{p}.$$

2) *Accurate Mode*: In the accurate mode, an additional INT8 complex matrix multiplication is performed in step 1, which also increases the memory traffic due to extra stores of INT8 matrices and loads of INT32 matrices, as well as additional loads and stores of the scaling vectors. The total execution time in this mode can therefore be estimated as

$$\frac{((35 + 3N + c)k + 8)(m + n) + (16N + 40 + 2c)mn}{b} + \frac{6(N + 1)mnk}{p},$$

and for the single-precision case,

$$\frac{((19 + 3N + c)k + 8)(m + n) + (16N + 32 + 2c)mn}{b} + \frac{6(N + 1)mnk}{p}.$$

Figs. 2 and 3 show performance-model heatmaps for the fast and accurate modes in single and double precision for 6 and 13 moduli, respectively. The TFLOPS was computed as

$$\frac{8mnk}{t} \cdot 10^{-12},$$

where t is the execution time in seconds. For example, assuming an effective memory bandwidth of 2–4 TByte/s and an observed INT8 GEMM kernel throughput of 1500 TFLOPS on the GH200, the model predicts that ZGEMM emulation in the accurate mode would achieve approximately 120 TFLOPS.

D. GEMM Emulation Library

In this study, we developed a library that provides a complete emulation environment for GEMM operations based on the Ozaki-II scheme. Details of the library and usage instructions are provided in [34]. The library supports both CUDA and HIP backends, and the CUDA-based source code is automatically translated into HIP C++ when compiled under the HIP backend. The implementation provides emulation routines for SGEMM and DGEMM as proposed in previous work, as well as the CGEMM and ZGEMM emulation kernels developed in the present study. The supported data types are float, double, cuComplex, cuDoubleComplex, hipFloatComplex, hipDoubleComplex.

To enable transparent integration with existing HPC applications, the library can be activated by adding the path of

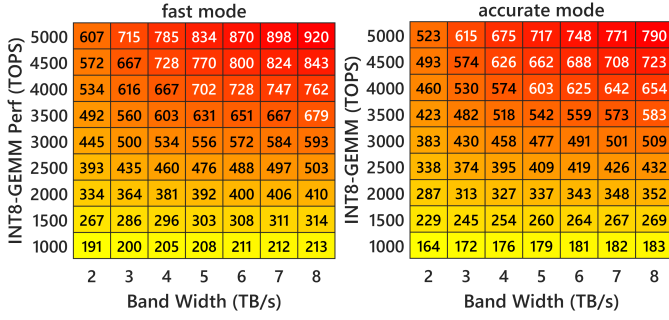


Fig. 2. Performance-model heatmaps for the single-precision complex matrix multiplication emulation. The left panel corresponds to the fast mode and the right panel to the accurate mode. The horizontal axis denotes the attainable memory bandwidth, and the vertical axis denotes the achievable INT8 GEMM throughput. The problem size is fixed to $m = n = k = 16384$, and the correction term is set to $c = 6$, equal to the number of moduli used in the emulation. The color scale indicates the predicted throughput in TFLOPS of the proposed emulation.

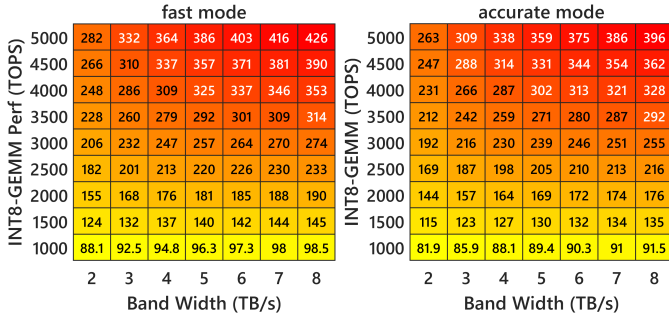


Fig. 3. Performance-model heatmaps for the double-precision complex matrix multiplication emulation. The left panel corresponds to the fast mode and the right panel to the accurate mode. The horizontal axis denotes the attainable memory bandwidth, and the vertical axis denotes the achievable INT8 GEMM throughput. The problem size is fixed to $m = n = k = 16384$, and the correction term is set to $c = 13$, equal to the number of moduli used in the emulation. The color scale indicates the predicted throughput in TFLOPS of the proposed emulation.

the shared library to the environment variable `LD_PRELOAD`. Once activated, the following GEMM calls to CUDA or HIP backends are dynamically redirected to the emulation routines, allowing users to evaluate GEMM emulation without modifying their applications:

- `cublasSgemm`, `cublasDgemm`
- `cublasCgemm`, `cublasZgemm`
- `cublasSgemm_v2`, `cublasDgemm_v2`
- `cublasCgemm_v2`, `cublasZgemm_v2`
- `cublasGemmEx`
- `hipblasSgemm`, `hipblasDgemm`
- `hipblasCgemm`, `hipblasZgemm`
- `hipblasGemmEx`, `hipblasGemmEx_v2`

The implementation consists of INT8 matrix multiplications, fixed-order basic arithmetic operations, and the evaluation of the $\log_2(\cdot)$ function appearing in (11), (12), (13), and (14). Rounding errors arise only in the computation of the scaling vectors defined by (11), (12), (13), and (14), and in the accumulation and final reduction stages of the

CRT. Except for the logarithm evaluation, all computations are deterministic sequences of basic arithmetic, and thus the emulation produces bitwise-reproducible results as long as the precision of the $\log_2(\cdot)$ computation is identical. When the API versions coincide, each backend employs identical implementations of `__log2f`, ensuring reproducibility across runs.

IV. NUMERICAL RESULTS

Here, we compare the following methods with each other:

Cgemm: `cublasCgemm` or `hipblasCgemm` using native FP32

Zgemm: `cublasZgemm` or `hipblasZgemm` using native FP64

Zgemm3m: `cublasZgemm3m` using native FP64

fast- N : Ozaki-II in fast mode with N moduli

accu- N : Ozaki-II in accurate mode with N moduli

OS I- S : Ozaki-I from cuBLAS in Fixed Mantissa Control mode with S slices

The hipBLAS does not provide `hipblasCgemm3m` or `hipblasZgemm3m` and the same limitation applies to the rocBLAS, though it is not used in this study. We have empirically confirmed that both the performance and numerical accuracy of `cublasCgemm3m` are essentially identical to those of `cublasCgemm`; thus, we omit its results from the following comparisons. The cuBLAS implementation of Ozaki-I-based emulation provides two execution modes: Dynamic Mantissa Control and Fixed Mantissa Control. In the Dynamic Mantissa Control mode, the routine determines the number of mantissa bits, equivalently, the number of slices, required for fixed-point emulation such that the numerical accuracy matches or exceeds that of FP64. If the required mantissa length exceeds either an internal limit or a user-specified maximum, the computation is automatically executed in native FP64. In the Fixed Mantissa Control mode, the user specifies the mantissa-bit length used for the fixed-point representation. This mode reduces runtime overhead by eliminating dynamic precision selection, but it does not ensure FP64-level accuracy. In this study, the Fixed Mantissa Control mode is used to enable a direct comparison of computational throughput and to quantify the performance gain attributable to fixed-precision emulation.

A. Accuracy

Each element of the test matrices $A \in \mathbb{C}^{m \times k}$ and $B \in \mathbb{C}^{k \times n}$ was generated by independently assigning its real and imaginary parts as

$$(\text{rand} - 0.5) \cdot \exp(\text{randn} \cdot \phi),$$

where $\text{rand} \in (0, 1]$ denotes a uniform random number and randn denotes a standard normal random number. Both types of random numbers were generated using either the cuRAND API or the hipRAND library. Following the methodology adopted in prior studies about the Ozaki schemes [23]–[25], [27]–[30], we introduce the parameter ϕ to control the dynamic range of the input matrices. In accordance with these studies, we specify $\phi \in \{0, 0.5, 1, 1.5\}$ for single-precision tests and $\phi \in \{0.5, 1, 2, 4\}$ for double-precision tests. As ϕ increases, the maximum magnitude grows exponentially, whereas the

median and interquartile range remain comparatively stable. This confirms that ϕ effectively widens the dynamic range without substantially affecting the central tendency of the distribution. Such controlled variation is essential for evaluating the performance and numerical stability of the Ozaki schemes.

Figs. 4 and 5 show the accuracy of CGEMM and ZGEMM, respectively, observed on the GH200. Similar results were obtained on other GPUs. The maximum relative error in the figures were calculated as

$$\max_{i,j} \left(\max \left(\frac{|\operatorname{Re}(\tilde{c}_{ij}) - \operatorname{Re}(c_{ij})|}{|\operatorname{Re}(c_{ij})|}, \frac{|\operatorname{Im}(\tilde{c}_{ij}) - \operatorname{Im}(c_{ij})|}{|\operatorname{Im}(c_{ij})|} \right) \right),$$

where $\tilde{C} \in \mathbb{C}^{m \times n}$ is the approximation of AB obtained using the emulation methods and the standard complex GEMM routines, and $C \in \mathbb{C}^{m \times n}$ is the high-precision result of AB computed using double-double arithmetic.

In CGEMM emulation, fast- N with $6 \leq N \leq 9$ and accu- N with $6 \leq N \leq 8$ provided the accuracy comparable with that of CGEMM. In ZGEMM emulation, fast- N with $13 \leq N \leq 18$ and accu- N with $13 \leq N \leq 17$ achieved the accuracy comparable with that of ZGEMM. As ϕ increases, the range of exponents for A and B expands, making it easier to overestimate the upper bound $2 \sum_{h=1}^k |a'_{ih}| |b'_{hj}|$ in the condition (4). DGEMM emulation in [30] required at least 14 moduli to achieve DGEMM-level accuracy, whereas the proposed method attains ZGEMM-level accuracy with only 13 moduli. This reduction is attributed to the use of the Karatsuba algorithm in the INT8 complex matrix multiplication, which suppresses precision loss more effectively than floating-point arithmetic.

The final reduction $\operatorname{mod}(s_{ij}, P)$ of the CRT is defined as $s_{ij} - P \cdot \operatorname{round}(s_{ij}/P)$. This formulation entails subtracting nearly identical quantities, leading to loss of significance. Therefore, obtaining the target accuracy requires that both the modulo operation and the calculation of s_{ij} be conducted at a precision level exceeding that of the final output. Regarding implementation, the limiting accuracy is currently constrained by the arithmetic precision employed: CGEMM emulation relies on double-precision arithmetic, while ZGEMM emulation calculates s_{ij} using the high-precision approach in (5) and employs a simplified high-precision method based on double-double arithmetic for the modulo operation. Enhancing the limiting accuracy therefore necessitates performing these operations with even higher precision.

B. Performance

Figs. 6–13 show the throughput performance of CGEMM and ZGEMM. For CGEMM, fast- N with $6 \leq N \leq 9$ and accu- N with $6 \leq N \leq 8$ achieved substantial speedups on the GH200 and B200 GPUs, reaching approximately $3.6\times$ – $5.4\times$ and $4.4\times$ – $6.5\times$, respectively. On the MI300X, however, the improvements were modest, with gains limited to $1.1\times$ – $1.6\times$. For ZGEMM, where fast- N spans $13 \leq N \leq 18$ and accu- N covers $13 \leq N \leq 17$, a similar trend was observed. The GH200 delivered moderate acceleration, achieving $1.7\times$ – $2.4\times$, whereas the B200 consistently attained the

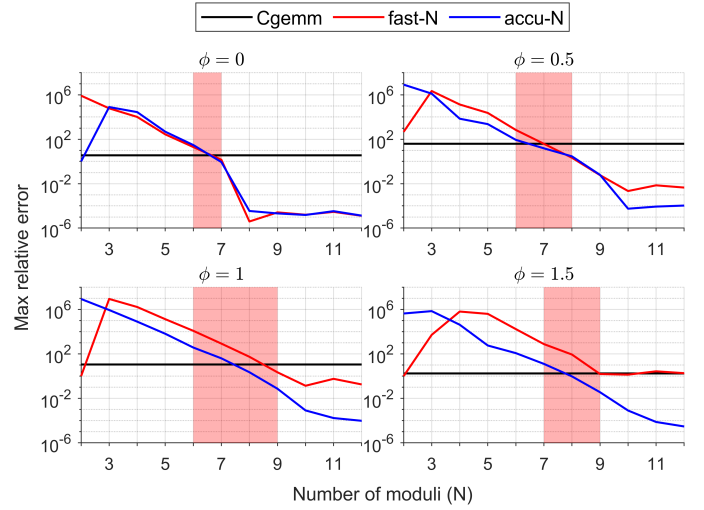


Fig. 4. Maximum relative error of single-precision complex matrix multiplication on NVIDIA GH200 Grace Hopper Superchip, CUDA Toolkit 13.0.88, and gcc 11.5.0, for $m = n = 1024$ and $k = 16384$.

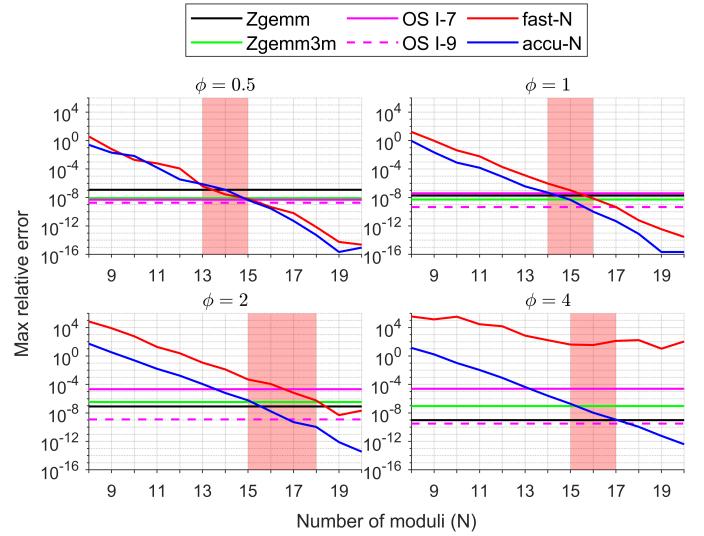


Fig. 5. Maximum relative error of double-precision complex matrix multiplication on NVIDIA GH200 Grace Hopper Superchip, CUDA Toolkit 13.0.88, and gcc 11.5.0, for $m = n = 1024$ and $k = 16384$.

highest speedups of $4.0\times$ – $5.6\times$. In contrast, the MI300X again exhibited only minimal improvement, ranging from $0.8\times$ to $1.1\times$, effectively matching or slightly underperforming the baseline routine. In addition, on the RTX 5080, we observed the following performance gains: for CGEMM, the proposed methods yielded $2.1\times$ – $3.2\times$; for ZGEMM, the proposed methods yielded $22\times$ – $30\times$. Overall, the Ozaki-II emulation yielded significant performance benefits on NVIDIA architectures, especially on B200 and RTX 5080, while demonstrating only limited impact on the MI300X.

These performance characteristics correlate strongly with the architectural balance of arithmetic throughput on each GPU. The GH200 provides a ratio of FP64 : FP32 : INT8 = 1 : 1 : 29.5, indicating that INT8 operations

TFLOPS on GH200					
fast-3	42.3	134.9	348.2	482.1	605.5
fast-4	34.9	109.9	280.3	367.5	449.5
fast-5	29.6	91.9	232.1	298.1	359.9
<i>fast-6</i>	25.4	78.0	196.1	249.3	299.4
<i>fast-7</i>	22.3	68.0	170.7	215.0	256.2
<i>fast-8</i>	20.0	61.0	152.6	188.1	225.1
<i>fast-9</i>	18.1	54.9	137.0	167.8	200.0
fast-10	16.5	49.8	123.9	151.4	181.0
fast-11	15.1	45.6	113.0	138.2	164.7
accu-3	34.7	102.7	259.2	355.9	448.3
accu-4	28.1	87.6	220.8	292.1	359.7
accu-5	25.4	75.6	189.4	247.2	300.2
<i>accu-6</i>	22.1	65.9	164.2	212.3	256.3
<i>accu-7</i>	19.2	58.7	145.5	186.3	223.7
<i>accu-8</i>	17.9	53.2	132.0	166.0	199.5
accu-9	16.0	48.6	120.5	150.0	180.1
accu-10	15.0	44.6	109.9	136.6	164.3
accu-11	13.6	41.1	101.1	125.1	150.6
Cgemm	48.8	52.6	54.0	55.5	55.8
	1024	2048	4096	8192	16384
$m = n = k$					

Fig. 6. Throughput performance in TFLOPS of single-precision complex matrix multiplication on NVIDIA GH200 Grace Hopper Superchip, CUDA Toolkit 13.0.88, and gcc 11.5.0, for $m = k = n$. Methods shown in red italic achieve CGEMM-level accuracy.

TFLOPS on B200					
fast-3	36.7	184.0	510.0	848.0	1000.0
fast-4	29.4	152.0	414.0	638.0	696.0
fast-5	24.3	129.0	351.0	498.0	562.0
<i>fast-6</i>	20.7	110.0	297.0	424.0	467.0
<i>fast-7</i>	18.1	97.0	259.0	364.0	398.0
<i>fast-8</i>	16.2	86.4	231.0	319.0	351.0
<i>fast-9</i>	14.5	78.1	206.0	285.0	312.0
fast-10	13.2	71.3	188.0	262.0	286.0
fast-11	12.1	65.3	170.0	238.0	261.0
accu-3	28.7	137.0	371.0	604.0	716.0
accu-4	23.6	119.0	321.0	493.0	565.0
accu-5	20.4	103.0	278.0	416.0	471.0
<i>accu-6</i>	17.5	90.7	241.0	361.0	399.0
<i>accu-7</i>	15.6	81.6	216.0	313.0	350.0
<i>accu-8</i>	14.2	74.2	196.0	285.0	312.0
accu-9	12.9	67.8	179.0	257.0	283.0
accu-10	11.9	62.7	165.0	236.0	261.0
accu-11	11.0	58.0	152.0	216.0	239.0
Cgemm	52.9	66.7	70.5	70.6	71.6
	1024	2048	4096	8192	16384
$m = n = k$					

Fig. 7. Throughput performance in TFLOPS of single-precision complex matrix multiplication on NVIDIA B200 Blackwell GPU with Intel Xeon Platinum 8570 CPU, CUDA Toolkit 12.8.93, and g++ 13.3.0, for $m = k = n$. Methods shown in red italic achieve CGEMM-level accuracy.

are moderately more abundant than FP64 and FP32 units. The B200 exhibits an even more pronounced imbalance, 1 : 2 : 121.6, where INT8 throughput overwhelmingly dominates the floating-point pipelines. This architectural skew directly amplifies the benefit of the proposed kernels, which replace large portions of floating-point arithmetic with INT8-intensive computation. In contrast, the MI300X offers a more subdued ratio of 1 : 1 : 16, providing substantially less INT8 advantage relative to its floating-point throughput. As a result, the computational substitution enabled by our method yields only marginal acceleration on the MI300X, consistent with the limited speedups observed in the benchmarks. For the RTX 5080, the observed performance gains, especially the very large speedups in ZGEMM, suggest a much stronger favoring of INT8 throughput relative to floating-point units.

TFLOPS on 5080					
fast-3	35.6	66.0	96.4		129.0
fast-4	27.7	51.5	73.7		95.6
fast-5	23.1	42.1	60.7		77.4
<i>fast-6</i>	19.8	35.7	51.4		64.8
<i>fast-7</i>	17.5	30.7	44.6		54.6
<i>fast-8</i>	15.6	27.4	39.2		47.3
<i>fast-9</i>	14.0	24.6	35.1		42.2
fast-10	12.6	22.3	31.7		38.3
fast-11	11.5	20.4	28.8		35.0
accu-3	28.3	49.4	72.6		96.1
accu-4	23.3	41.0	59.5		77.4
accu-5	19.8	34.8	50.5		65.0
<i>accu-6</i>	17.3	30.3	43.9		55.9
<i>accu-7</i>	15.4	26.9	38.6		48.2
<i>accu-8</i>	13.9	24.1	34.7		42.4
accu-9	12.7	21.9	31.4		38.5
accu-10	11.7	20.1	28.7		35.1
accu-11	10.7	18.5	26.2		32.3
Cgemm	23.7	22.4	20.3		20.0
	1024	2048	4096		8192
$m = n = k$					

Fig. 8. Throughput performance in TFLOPS of single-precision complex matrix multiplication on NVIDIA GeForce RTX 5080 GPU with AMD Ryzen 7 3700X CPU, CUDA Toolkit 13.0.88, and gcc 11.5.0, for $m = k = n$. Methods shown in red italic achieve CGEMM-level accuracy.

TFLOPS on MI300X					
fast-3	8.3	53.4	197.8	297.9	343.2
fast-4	6.6	42.5	152.5	219.6	243.4
fast-5	5.6	35.5	124.8	176.9	195.7
<i>fast-6</i>	4.9	29.6	106.0	148.8	162.4
<i>fast-7</i>	4.3	25.2	91.7	127.8	140.2
<i>fast-8</i>	3.8	21.9	81.0	112.3	122.2
<i>fast-9</i>	3.4	19.5	73.2	100.4	109.6
fast-10	3.1	17.7	65.6	91.6	99.3
fast-11	2.9	16.1	59.7	83.1	91.4
accu-3	6.5	43.2	153.6	221.9	251.0
accu-4	5.5	36.0	125.1	176.9	197.8
accu-5	4.7	31.0	106.3	149.4	165.6
<i>accu-6</i>	4.2	26.0	92.1	127.9	141.3
<i>accu-7</i>	3.7	22.8	81.7	112.6	124.1
<i>accu-8</i>	3.4	20.1	72.9	99.9	109.8
accu-9	3.1	18.1	65.9	91.2	100.1
accu-10	2.9	16.4	59.9	83.5	91.5
accu-11	2.6	15.1	54.9	76.4	84.5
Cgemm	48.8	94.3	116.7	96.7	99.9
	1024	2048	4096	8192	16384
$m = n = k$					

Fig. 9. Throughput performance in TFLOPS of single-precision complex matrix multiplication on AMD Instinct MI300X GPU with AMD EPYC 9534 CPU, ROCm 6.4.2, and clang 19.0.0git, for $m = k = n$. Methods shown in red italic achieve CGEMM-level accuracy.

The architectural balance on RTX 5080 corresponds to FP64 : FP32 : INT8 = 1 : 64 : 512, which explains the pronounced acceleration when using INT8-based emulation.

Compared with the Ozaki-I emulation implemented in cuBLAS, the proposed method delivered substantially higher throughput on both GH200 and RTX 5080 GPUs. On the GH200, Ozaki-I with 7–9 slices achieved 25–39 TFLOPS, whereas the proposed method attained 99–136 TFLOPS. On the RTX 5080, Ozaki-I achieved 6.4–10 TFLOPS, while the proposed method reached 19–26 TFLOPS. These disparities arise from the fundamental algorithmic differences: Ozaki-I with S slices requires $S(S+1)/2$ INT8 complex matrix multiplications, whereas the proposed technique with N moduli requires only N INT8 complex matrix multiplications and further reduces their cost by a factor of 3/4 using the

TFLOPS on GH200					
fast-8	19.3	58.7	147.0	184.0	221.0
fast-9	17.6	52.9	132.0	164.0	197.0
fast-10	16.1	48.3	120.0	147.0	177.0
fast-11	14.8	44.3	110.0	134.0	161.0
fast-12	13.7	41.1	102.0	123.0	149.0
<i>fast-13</i>	12.4	36.9	91.6	112.0	136.0
<i>fast-14</i>	11.6	34.3	85.1	104.0	127.0
<i>fast-15</i>	10.9	32.1	79.5	96.6	118.0
<i>fast-16</i>	10.3	30.3	74.8	90.5	111.0
<i>fast-17</i>	9.8	28.6	70.5	85.6	105.0
<i>fast-18</i>	9.2	27.1	66.6	80.8	99.0
fast-19	8.6	25.0	61.4	75.7	93.3
fast-20	8.2	23.7	58.5	72.0	88.8
accu-8	17.1	51.2	126.0	162.0	196.0
accu-9	15.7	46.9	115.0	147.0	176.0
accu-10	14.5	43.1	106.0	133.0	161.0
accu-11	13.5	39.9	98.0	122.0	148.0
accu-12	12.6	37.3	91.6	112.0	137.0
<i>accu-13</i>	11.5	33.7	82.6	103.0	126.0
<i>accu-14</i>	10.8	31.6	77.4	95.7	117.0
<i>accu-15</i>	10.2	29.7	72.7	89.5	110.0
<i>accu-16</i>	9.7	28.2	68.8	84.7	104.0
<i>accu-17</i>	9.2	26.7	65.2	80.1	98.5
accu-18	8.7	25.4	61.6	75.8	93.5
accu-19	8.2	23.4	57.0	71.3	88.3
accu-20	7.8	22.4	54.6	68.0	84.2
OS I-7	17.2	26.7	29.7	30.0	39.0
OS I-8	14.6	21.8	23.6	23.6	30.7
OS I-9	12.6	18.2	19.2	19.0	24.8
Zgemm	52.7	58.9	57.8	57.6	57.4
Zgemm3m	58.2	65.1	65.2	64.6	64.5
1024 2048 4096 8192 16384					
m = n = k					

Fig. 10. Throughput performance in TFLOPS of double-precision complex matrix multiplication on NVIDIA GH200 Grace Hopper Superchip, CUDA Toolkit 13.0.88, and gcc 11.5.0, for $m = k = n$. Methods shown in red italic achieve ZGEMM-level accuracy.

TFLOPS on RTX 5080				
fast-8	9.2	18.2	29.3	41.6
fast-9	8.2	16.3	26.2	37.1
fast-10	7.5	14.8	23.7	33.6
fast-11	6.9	13.5	21.6	30.5
fast-12	6.4	12.5	19.9	28.1
<i>fast-13</i>	5.9	11.5	18.3	25.8
<i>fast-14</i>	5.5	10.7	17.0	24.1
<i>fast-15</i>	5.2	10.0	16.0	22.5
<i>fast-16</i>	4.9	9.4	15.0	21.1
<i>fast-17</i>	4.6	8.9	14.1	19.9
<i>fast-18</i>	4.4	8.4	13.4	18.8
fast-19	4.1	8.0	12.7	17.8
fast-20	3.9	7.6	12.1	17.0
accu-8	8.7	16.7	26.5	37.3
accu-9	7.8	15.1	24.0	33.7
accu-10	7.2	13.8	21.9	30.7
accu-11	6.6	12.7	20.1	28.3
accu-12	6.1	11.8	18.6	26.1
<i>accu-13</i>	5.7	10.9	17.2	24.1
<i>accu-14</i>	5.3	10.2	16.1	22.6
<i>accu-15</i>	5.0	9.6	15.1	21.2
<i>accu-16</i>	4.7	9.0	14.2	20.0
<i>accu-17</i>	4.3	8.5	13.5	18.9
accu-18	4.1	8.1	12.8	17.9
accu-19	4.0	7.7	12.2	17.0
accu-20	3.8	7.4	11.6	16.2
OS I-7	6.2	8.9	10.0	10.3
OS I-8	5.1	7.2	8.0	8.0
OS I-9	4.3	5.9	6.5	6.4
Zgemm	0.8	0.8	0.8	0.8
Zgemm3m	1.0	1.1	1.1	1.1
1024 2048 4096 8192				
m = n = k				

Fig. 12. Throughput performance in TFLOPS of double-precision complex matrix multiplication on NVIDIA GeForce RTX 5080 GPU with AMD Ryzen 7 3700X CPU, CUDA Toolkit 13.0.88, and gcc 11.5.0, for $m = k = n$. Methods shown in red italic achieve ZGEMM-level accuracy.

TFLOPS on B200					
fast-8	15.8	83.5	223.0	316.0	347.0
fast-9	14.2	75.8	201.0	283.0	310.0
fast-10	13.0	69.4	184.0	255.0	282.0
fast-11	11.9	63.9	168.0	234.0	256.0
fast-12	11.0	59.5	156.0	218.0	231.0
<i>fast-13</i>	9.9	52.7	138.0	190.0	209.0
<i>fast-14</i>	9.4	49.4	128.0	177.0	194.0
<i>fast-15</i>	8.8	46.3	120.0	165.0	180.0
<i>fast-16</i>	8.3	43.7	113.0	155.0	169.0
<i>fast-17</i>	7.9	41.4	106.0	146.0	158.0
<i>fast-18</i>	7.5	39.2	101.0	139.0	149.0
fast-19	7.0	35.8	91.3	127.0	138.0
fast-20	6.7	34.1	87.2	121.0	130.0
accu-8	13.9	71.9	191.0	276.0	308.0
accu-9	12.7	65.9	175.0	250.0	278.0
accu-10	11.7	61.2	161.0	226.0	252.0
accu-11	10.8	56.7	150.0	212.0	233.0
accu-12	10.1	53.3	140.0	193.0	213.0
<i>accu-13</i>	9.3	47.7	124.0	175.0	193.0
<i>accu-14</i>	8.7	44.9	117.0	165.0	181.0
<i>accu-15</i>	8.2	42.4	110.0	155.0	170.0
<i>accu-16</i>	7.7	40.2	104.0	145.0	158.0
<i>accu-17</i>	7.4	38.2	98.1	138.0	147.0
accu-18	7.0	36.3	93.5	130.0	141.0
accu-19	6.6	33.4	85.0	120.0	130.0
accu-20	6.3	32.0	81.5	115.0	124.0
Zgemm	31.3	36.0	36.8	36.7	37.1
Zgemm3m	39.4	43.9	44.6	45.2	45.3
1024 2048 4096 8192 16384					
m = n = k					

Fig. 11. Throughput performance in TFLOPS of double-precision complex matrix multiplication on NVIDIA B200 Blackwell GPU with Intel Xeon Platinum 8570 CPU, CUDA Toolkit 12.8.93, and g++ 13.3.0, for $m = k = n$. Methods shown in red italic achieve ZGEMM-level accuracy.

TFLOPS on MI300X				
fast-8	3.7	20.8	79.2	110.0
fast-9	3.3	18.8	70.9	98.5
fast-10	3.0	17.2	64.5	89.2
fast-11	2.8	15.8	58.9	81.0
fast-12	2.6	14.5	53.9	74.1
<i>fast-13</i>	2.4	13.6	50.2	68.7
<i>fast-14</i>	2.2	12.7	46.5	63.8
<i>fast-15</i>	2.1	11.8	43.5	59.5
<i>fast-16</i>	2.0	11.1	41.2	55.9
<i>fast-17</i>	1.9	10.4	38.7	53.0
<i>fast-18</i>	1.8	9.8	36.7	50.2
fast-19	1.7	9.4	34.2	47.5
fast-20	1.6	8.9	32.4	45.1
accu-8	3.3	19.5	71.1	98.5
accu-9	3.0	17.6	64.9	89.1
accu-10	2.8	16.1	59.3	81.2
accu-11	2.6	14.8	54.2	74.5
accu-12	2.4	13.7	50.2	68.5
<i>accu-13</i>	2.3	12.9	46.6	63.6
<i>accu-14</i>	2.1	12.0	43.3	59.5
<i>accu-15</i>	2.0	11.3	40.7	55.7
<i>accu-16</i>	1.9	10.7	38.4	52.5
<i>accu-17</i>	1.8	10.0	36.5	50.0
accu-18	1.7	9.5	34.2	47.4
accu-19	1.6	9.1	32.6	45.0
accu-20	1.6	8.6	30.5	42.8
Zgemm	39.5	51.8	83.1	70.0
1024 2048 4096 8192 16384				
m = n = k				

Fig. 13. Throughput performance in TFLOPS of double-precision complex matrix multiplication on AMD Instinct MI300X GPU with AMD EPYC 9534 CPU, ROCm 6.4.2, and clang 19.0.0git, for $m = k = n$. Methods shown in red italic achieve ZGEMM-level accuracy.

Karatsuba method. This reduction in both the number and cost of INT8 matrix multiplication kernel invocations directly explains the substantially higher performance of the proposed method.

C. Supplemental Comparison for Real-Valued Emulation

Here, we focus on double-precision real matrix multiplication. In this section, we focus on double-precision real matrix multiplication. We compare two emulation strategies: the Ozaki-I based emulation provided by cuBLAS and the Ozaki-II based emulation proposed in [30], enhanced with the blocking technique introduced in Section III. This supplemental analysis aims to isolate the behavior of Ozaki-type schemes in the real domain and to clarify the performance implications of incorporating blocking into Ozaki-II under conditions consistent with real-valued DGEMM emulation.

According to [30], the fast mode and the accurate mode require 14–18 and 14–17 moduli, respectively, to achieve DGEMM-level accuracy. On the GH200 with matrix size $m = n = k = 16384$, the Ozaki-II fast- N configuration with $14 \leq N \leq 18$ achieved 63–82 TFLOPS, while the accu- N configuration with $14 \leq N \leq 17$ reached 63–75 TFLOPS. Incorporating the blocking technique improved performance across all settings: the blocked fast- N version attained 72–93 TFLOPS, and the blocked accu- N version delivered 71–85 TFLOPS. cuBLAS’s Ozaki-I implementation with 7–10 slices achieved only 20–39 TFLOPS, and the standard DGEMM kernel sustained 61 TFLOPS.

On the RTX 5080 with $m = n = k = 8192$, the fast- N and accu- N configurations of the proposed method reached 12–16 TFLOPS and 12–15 TFLOPS, respectively, while cuBLAS’s Ozaki-I achieved 5.4–10 TFLOPS. In comparison, the native DGEMM kernel delivered 0.84 TFLOPS, highlighting the substantial advantage of INT8-intensive Ozaki-type emulation on this consumer architecture.

These results demonstrate that blocking substantially enhances the performance of Ozaki-II for large-scale problems, particularly on bandwidth-rich GPUs such as the H200, and Ozaki-II, whether blocked or unblocked, consistently outperforms cuBLAS’s Ozaki-I due to its reduced number of INT8 matrix multiplication invocations.

V. CONCLUSION

This study proposed emulation methods of complex matrix multiplication based on the Ozaki-II scheme and developed a portable library that supports execution on both AMD and NVIDIA GPUs. The numerical results demonstrated that the proposed methods outperform the standard GEMM routines in cuBLAS and hipBLAS, which are implemented using native FP32 and FP64 arithmetic. In addition, the proposed methods outperform the emulation using the Ozaki-I scheme implemented in cuBLAS.

Future work includes comparisons with the BF16x9 CGEMM emulation algorithm provided for NVIDIA B200 and B300 GPUs in cuBLAS 12.9 and later. A major limitation of the proposed method, as well as emulation-based approaches

in general, is the substantial working memory required. This overhead is currently unavoidable when aiming to achieve high performance without relying on FP32 or FP64 matrix multiplication. Addressing this issue remains an open challenge not only for emulation techniques but also for HPC applications that employ them.

The Ozaki-I and Ozaki-II schemes utilize INT8 matrix engines, benefiting from the fact that the error-free low-precision matrix multiplications appearing in these schemes do not require exponent handling, enabling efficient synergy with INT8 operations with INT32 accumulation. When using FP16 or BF16 matrix engines with FP32 accumulation, the effective representable integer range is limited to 24 bits due to the FP32 significand, leading to significantly reduced computational efficiency compared with INT32 accumulation. However, some recent GPUs, such as the NVIDIA B300, provide reduced INT8 capabilities. Supporting such hardware architectures represents an additional direction for future work.

ACKNOWLEDGMENT

We thank Prof. Rio Yokota for his cooperation in conducting the experiments on the B200. This study was supported by Japan Society for the Promotion of Science Grant-in-Aid Numbers 25K03126 (Scientific Research B) and 24K23874 (Research Activity Start-up).

REFERENCES

- [1] NVIDIA Corporation. (2025) NVIDIA Tensor Cores. Retrieved 25 July, 2025. [Online]. Available: <https://www.nvidia.com/en-us/data-center/tensor-cores/>
- [2] Advanced Micro Devices, Inc. (2025) AMD matrix cores. Retrieved 21 March, 2025. [Online]. Available: <https://rocm.blogs.amd.com/software-tools-optimization/matrix-cores/README.html>
- [3] Google LLC. (2025) Cloud Tensor Processing Unit. Retrieved 21 March, 2025. [Online]. Available: <https://cloud.google.com/tpu/docs/intro-to-tpu>
- [4] R. Perumal, N. Chinthapani, and R. D’Souza. (2024) Heterogeneous ai powerhouse: Unveiling the hardware and software foundation of intel® core™ ultra processors for the edge, white paper. [Online]. Available: <https://www.intel.com/content/www/us/en/content-details/817736>
- [5] Advanced Micro Devices, Inc. (2025) Amd instinct mi300x apu. Retrieved 5 December, 2025. [Online]. Available: <https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/data-sheets/amd-instinct-mi300a-data-sheet.pdf>
- [6] —. (2025) Amd instinct mi300x accelerator. Retrieved 5 December, 2025. [Online]. Available: <https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/data-sheets/amd-instinct-mi300x-data-sheet.pdf>
- [7] —. (2025) Amd instinct mi325x accelerator. Retrieved 5 December, 2025. [Online]. Available: <https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/product-briefs/instinct-mi325x-datasheet.pdf>
- [8] —. (2025) Amd instinct mi350x gpu. Retrieved 5 December, 2025. [Online]. Available: <https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/product-briefs/amd-instinct-mi350x-gpu-brochure.pdf>
- [9] —. (2025) Amd instinct mi355x gpu. Retrieved 5 December, 2025. [Online]. Available: <https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/product-briefs/amd-instinct-mi355x-gpu-brochure.pdf>
- [10] NVIDIA Corporation. (2020) Nvidia a100 tensor core gpu architecture v1.0. Retrieved 5 December, 2025. [Online]. Available: <https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf>

- [11] ——. (2023) Nvidia h100 tensor core gpu architecture v1.04. Retrieved 5 December, 2025. [Online]. Available: <https://resources.nvidia.com/en-us-hopper-architecture/nvidia-h100-tensor-c>
- [12] ——. (2025) Nvidia blackwell architecture technical brief v2.1. Retrieved 5 December, 2025. [Online]. Available: <https://resources.nvidia.com/en-us-blackwell-architecture>
- [13] ——. (2025) Nvidia rtx blackwell gpu architecture v1.1. Retrieved 5 December, 2025. [Online]. Available: <https://images.nvidia.com/aem-dam/Solutions/geforce/blackwell/nvidia-rtx-blackwell-gpu-architecture.pdf>
- [14] Microsoft. (2025) Copilot+ PCs. Retrieved 27 November, 2025. [Online]. Available: <https://www.microsoft.com/en-us/windows/copilot-plus-pcs>
- [15] Z. Xiu. (2025) Part 1: Arm scalable matrix extension (sme) introduction. Retrieved 27 November, 2025. [Online]. Available: <https://developer.arm.com/community/arm-community-blogs/b/architectures-and-processors-blog/posts/arm-scalable-matrix-extension-introduction>
- [16] C. Brower, S. R. Bernabeu, J. Hammond, J. Gunnels, S. S. Xanthea, M. Ganahl, A. Menczer, and Örs Legeza, "Mixed-precision ab initio tensor network state methods adapted for nvidia blackwell technology via emulated fp64 arithmetic," 2025. [Online]. Available: <https://arxiv.org/abs/2510.04795>
- [17] A. Abdelfattah, J. Dongarra, M. Fasi, M. Mikaitis, and F. Tisseur, "Analysis of floating-point matrix multiplication computed via integer arithmetic," 2025. [Online]. Available: <https://arxiv.org/abs/2506.11277>
- [18] A. Schwarz, A. Anders, C. Brower, H. Bayraktar, J. Gunnels, K. Clark, R. G. Xu, S. Rodriguez, S. Cayrols, P. Tabaszewski, and V. Podlozhnyuk, "Guaranteed dgemm accuracy while using reduced precision tensor cores through extensions of the ozaki scheme," 2025. [Online]. Available: <https://arxiv.org/abs/2511.13778>
- [19] D. Mukunoki and K. Ozaki, "Sparse iterative solvers using high-precision arithmetic with quasi multi-word algorithms," 2025. [Online]. Available: <https://arxiv.org/abs/2510.13536>
- [20] J. Dongarra, J. Gunnels, H. Bayraktar, A. Haidar, and D. Ernst, "Hardware trends impacting floating-point computations in scientific applications," 2024. [Online]. Available: <https://arxiv.org/abs/2411.12090>
- [21] P. Luszczyk, V. Gadepally, L. Anderson, W. Arcand, D. Bestor, W. Bergeron, A. Bonn, D. J. Burrill, C. Byun, M. Houle, M. Hubbell, M. Jones, P. Michaleas, G. Morales, J. Mullen, A. Prout, A. Reuther, A. Rosa, C. Yee, and J. Kepner, "Performance and numerical aspects of decomposition factorizations with fp64 floating-point emulation in int8," in *2025 IEEE High Performance Extreme Computing Conference (HPEC)*, 2025, pp. 1–7.
- [22] J. Dongarra, J. Gunnels, H. Bayraktar, A. Haidar, and D. Ernst, "Accelerating supercomputing: Ai-hardware-driven innovation for speed and efficiency," in *2025 IEEE High Performance Extreme Computing Conference (HPEC)*, 2025, pp. 1–7.
- [23] K. Ozaki, T. Ogita, S. Oishi, and S. M. Rump, "Error-free transformations of matrix multiplication by using fast routines of matrix multiplication and its applications," *Numerical Algorithms*, vol. 59, no. 1, pp. 95–118, 2012.
- [24] —, "Generalization of error-free transformation for matrix multiplication and its application," *Nonlinear Theory and Its Applications, IEICE*, vol. 4, no. 1, pp. 2–11, 2013.
- [25] D. Mukunoki, K. Ozaki, T. Ogita, and T. Imamura, "Dgemm using tensor cores, and its accurate and reproducible versions," in *High Performance Computing*, P. Sadayappan, B. L. Chamberlain, G. Juckeland, and H. Ltaief, Eds. Cham: Springer International Publishing, 2020, pp. 230–248.
- [26] D. Mukunoki, "Dgemm without fp64 arithmetic - using fp64 emulation and fp8 tensor cores with ozaki scheme," 2025. [Online]. Available: <https://arxiv.org/abs/2508.00441>
- [27] H. Ootomo, K. Ozaki, and R. Yokota, "Dgemm on integer matrix multiplication unit," *The International Journal of High Performance Computing Applications*, vol. 38, no. 4, pp. 297–313, 2024.
- [28] Y. Uchino, K. Ozaki, and T. Imamura, "Performance enhancement of the ozaki scheme on integer matrix multiplication unit," *The International Journal of High Performance Computing Applications*, vol. 39, no. 3, pp. 462–476, 2025.
- [29] K. Ozaki, Y. Uchino, and T. Imamura, "Ozaki Scheme II: A GEMM-oriented emulation of floating-point matrix multiplication using an integer modular technique," 2025.
- [30] Y. Uchino, K. Ozaki, and T. Imamura, "High-performance and power-efficient emulation of matrix multiplication using int8 matrix engines," in *Proceedings of the SC '25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC Workshops '25. St. Louis, MO, USA: Association for Computing Machinery, 2025, pp. 1824–1831. [Online]. Available: <https://doi.org/10.1145/3731599.3767539>
- [31] NVIDIA Corporation, "CUDA C++ Programming Guide Release 13.0," 2025, retrieved 7 October, 2025. [Online]. Available: <https://docs.nvidia.com/cuda/cuda-c-programming-guide>
- [32] J.-M. Muller, "On the definition of ulp(x)," INRIA, LIP, Research Report RR-5504, LIP RR-2005-09, Feb. 2005. [Online]. Available: <https://inria.hal.science/inria-00070503>
- [33] Advanced Micro Devices, Inc., "HIP math API — HIP 7.0.51831 Documentation," 2025, retrieved 7 October, 2025. [Online]. Available: https://rocm.docs.amd.com/projects/HIP/en/latest/reference/math_api.html
- [34] Y. Uchino, "GEMMu8: GEMM emulation using int8 matrix engines based on the Ozaki scheme II," R-CCS GitHub repository, 2025. [Online]. Available: <https://github.com/RIKEN-RCCS/GEMMu8>